

Felipe Lemos

Software Engineer | Backend & Systems | Security Engineering | Rust, Python, TypeScript, C/C++ | Roblox Developer

Rio de Janeiro, Brazil · <https://www.linkedin.com/in/yuee/> · <https://github.com/edtw> · <https://www.roblox.com/pt/games/84720327590325/NEXUS>

SUMMARY

I'm a software engineer in Rio de Janeiro who learns systems by building them — and breaking them in an isolated lab. I work across backend services, Rust runtimes with local AI, release infrastructure, distributed-systems research, and Roblox gameplay, currently developing combat systems on NEXUS. I do my best work where ownership is real: small teams, hard constraints, and code that has to earn trust. Preparing for OSCP and open to software...

TECHNICAL SKILLS

- **Backend & product engineering:** TypeScript · Python · Node.js · FastAPI · React · Next.js · REST · GraphQL · Socket.IO
- **Systems & infrastructure:** Rust · C/C++ · Docker · CI/CD · PostgreSQL · Redis · SQLx · Prometheus
- **Security engineering:** Reverse engineering · AppSec · network analysis · cryptography · authorized testing
- **AI & interactive technology:** Candle · GGUF · TinyML · PyTorch · OpenCV · Roblox Studio · Luau

EXPERIENCE

Roblox Studio Developer, NEXUS (Roblox Studio)

Current

Gameplay development for a live PvP arena experience. Contributing to NEXUS in Roblox Studio, an evolving combat experience centered on distinctive characters, abilities, visual effects, and strategic player interaction. The role brings together gameplay logic, rapid iteration, technical collaboration, and attention to how systems feel in play.

SELECTED PROJECTS

Karma Network · Spirix: RUST · DISTRIBUTED SYSTEMS · MESH / BLOCKCHAIN / COMPUTE

Mesh network framework in Rust built around Proof of Antiquity blockchain, distributed computing, and agent capabilities. Three layers: blockchain (PoA consensus, Ed25519 wallets, hardware attestation, anti-Sybil nullifiers, on-chain marketplace, eclipse defense), mesh network (DHT overlay, mesh routing, relay forwarding, DAG state sync, peer discovery), and distributed compute (federated...

- Contribute to a large Rust workspace unifying blockchain, mesh networking, and distributed compute
- Work across DHT overlay, DAG state sync, relay services, and federated-learning primitives
- Integrate daily across site, API, agents, and docs in a fast-moving monorepo

Rust 2024 workspace · Tokio · Axum · SQLx/PostgreSQL · Candle · DHT/mesh · PoA chain · Docker

Micro Runtime: RUST · SYSTEMS ENGINEERING · LOCAL AI RUNTIME

Modular Rust 2021 runtime for constrained environments. Feature-gated architecture with optional no_std, custom allocator (4 KiB pages, sharded heap, ephemeral scopes), fixed 256-slot one-shot SecureDispatcher, read-only memory-mapped IO, closed HostOp ISA with session workers, and automated architecture validation in CI. Local AI execution model: TokenEngine trait decoupling numeric decisions...

- Built a feature-gated Rust runtime where capabilities compile in or out without forking the codebase
- Added local AI paths: memory-mapped GGUF loading and a no-heap TinyML VM with hard budgets
- Keep the architecture honest with CI inventories and size gates that fail dirty builds

Rust 2021 · no_std · Candle · GGUF · M35 VM · MRE1 · Wasm · Axum · cryptography · CI architecture gates

ESTH: Enterprise Scripting & Telemetry Host: C++ · PYTHON · LICENSED SYSTEMS AGENT

Licensed client + agent + cloud API for scripting and telemetry against an authorized first-party game client in isolated lab VMs. Four-plane separation: FastAPI cloud (license, device binding, ownership-gated sync, catalog, encrypted OffsetBundle v3, telemetry ingest), native loader GUI (login, HWID, catalog, sync, editor, agent supervision), local agent service (execution queue, data-driven...

- Built a cross-platform systems agent with local IPC, queued scheduling, and data-driven config sync
- Shipped license/device binding with ownership-gated, encrypted configuration delivery
- Backed it with integration tests, snapshot diagnostics, and docs written for fail-closed behavior

C++17 · CMake · FastAPI · Pydantic · AES-GCM · versioned JSON schemas · pytest integration tests · manual lab battery

Marambaia PDV: FULL STACK · REAL TIME · OPERATIONS

Multi-surface restaurant operations platform covering products, tables, orders, cash registers, customer QR ordering, analytics, backups, access control, and real-time synchronization. Includes work on race conditions and order-total consistency.

- Built real-time ordering, QR customer flows, and cash/table management for a restaurant POS
- Hunted down race conditions and order-total bugs across concurrent admin and customer surfaces

React · Node.js · Express · MongoDB · Socket.IO · JWT · transaction consistency

Prisma Platform: AI SERVICES · AUTOMATION · PLATFORM ARCHITECTURE

Service-oriented platform for WhatsApp connectivity and AI-assisted workflows. Separates web application, messaging service, AI handling, provider abstraction with fallback, intent detection, analytics, persistence, caching, and containerized operations.

- Designed a multi-service WhatsApp/AI platform with provider fallback and session management
- Drew hard lines between web, messaging, AI, persistence, and cache behind Docker Compose

Next.js · TypeScript · Express · FastAPI · PostgreSQL · Redis · Docker

Airshipper: RUST · RELEASE ENGINEERING · CROSS-PLATFORM

Customized desktop launcher and release service with platform artifact mapping, GitHub release integration, metrics, database-backed metadata, installers, and multi-platform build pipelines. Based on the open-source Veloren Airshipper; customization only, with upstream attribution.

- Customized the open-source Veloren launcher for GitHub-hosted release channels
- Mapped platform artifacts, metrics, and metadata with automated multi-OS packaging

Rust · Iced · Tokio · Axum · SQLx · Prometheus · release automation

SECURITY RESEARCH

Hands-on security research conducted for education, defense, and authorized testing only. Work emphasizes understanding operating-system behavior, binary and protocol analysis, and defensive detection — not operational tooling. Sensitive implementations stay...

- Windows internals and binary analysis in isolated VMs — studying telemetry to learn what defenders should log
- Protocol and app-behavior review, including hypothesis-driven API assessments under explicit authorization
- Malware-analysis fundamentals: loader telemetry, encrypted-IPC patterns, persistence indicators, control-plane detections
- Detection-engineering habit: reproduce suspicious behavior safely, then write the alert I wish existed

CERTIFICATIONS

- OSCP (Offensive Security Certified Professional) — in progress, self-study with lab practice
- TryHackMe — active learner, ongoing rooms across red-team and blue-team tracks

LANGUAGES & INTERESTS

Portuguese (native) · English (professional working proficiency)

Game development (Roblox Studio) · CTF challenges · home-lab security research · open-source Rust

NOTE

Sensitive repositories are described as private case studies without operational code. Architectural detail available in technical interviews where appropriate. Updated 2026-09-04 · v2.0.0